C Programming

Ch.7 Loops

while, do-while, for

Repetition (Loop)

1. Why Use a Loop?



Repeating the same action multiple times can be done efficiently using loops.

2. From Repetition to Loop

Without loop (repetition)

```
printf ( "Hello World! \n" );
printf ( "Hello World! \n" );
printf ( "Hello World! \n" );
printf ( "Hello World! \n" );
printf ( "Hello World! \n" );
```

With loop (using for)

```
for (i = 0; i < 5; i++)
{
   printf ( "Hello World! \n" );
}
```

A loop performs the same task repeatedly with fewer lines of code.

Key Points

- Loops allow a block of code to be executed repeatedly.
- `for` and `while` are the most common loop structures in C.
- Use loops when the number of repetitions is known or controlled by a condition.
- They reduce code size, improve readability, and help avoid repetition.
- Make sure the loop has a way to stop (update variable / condition) to avoid infinite loops.

Repetition structure: Type of Loop

• Repeat structure: a structure that loops until a certain condition is satisfied. C provides three: while, do-while, for.

for loop

```
for (int i = 0; i < n; i++)
{
    // code block
}
```

- Used when the number of repetitions is known.
- Initialization, condition, and update are in one line.

while loop

```
while (condition)
{
    // code block
}
```

- Used when the number of repetitions is not known.
- The condition is checked before the loop starts.

do-while loop

```
do
{
    // code block
} while (condition);
```

- Used when the code block must run at least once.
- The condition is checked after the loop.

while Loop

Syntax while statement

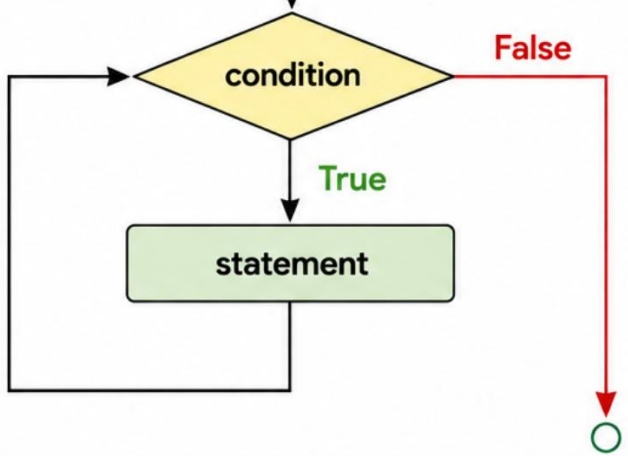
C

```c
while ( i < 10 ) {
    printf("Hello World!\n");
    i++;
}
```

the condition
(evaluated before the loop runs)

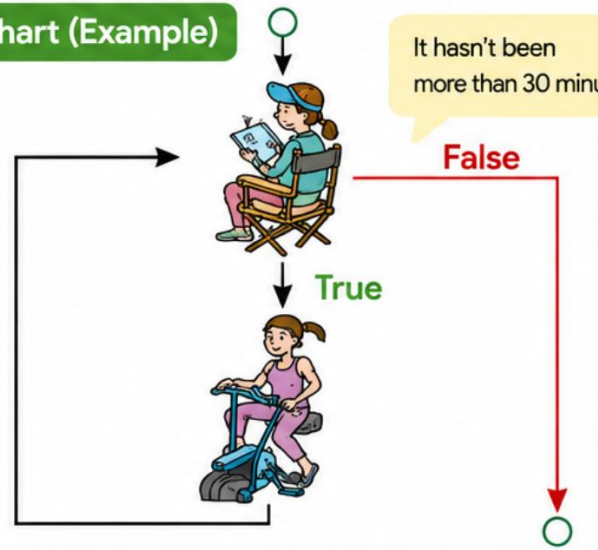
the code to be repeated
(executes if condition is True)

Flowchart (Basic)



- If the condition is True, the code (statement) runs and returns to check again.
- If the condition is False, the loop ends and the program continues.

Flowchart (Example)



- While the time is 30 minutes or less (True), keep reading and exercising.
- If the time exceeds 30 minutes (False), stop and move on.

while Loop

1. while Loop Example (C Code)

```c
#include <stdio.h>
int main(void)
{
    int i = 0;

    while (i < 5) {
        printf("Hello World!\n");
        i++;
    }

    return 0;
}
```

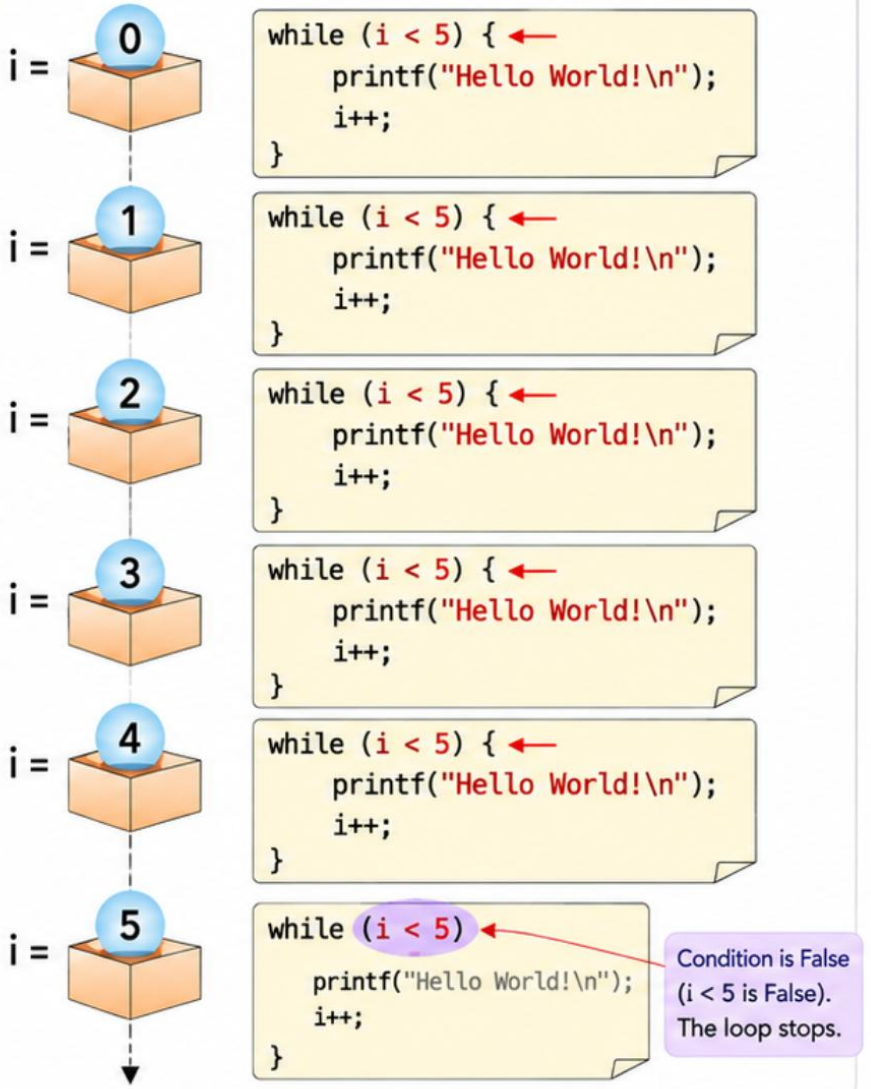
repeat condition (checked before each iteration)

repeat content (executed if condition is True)

How it works

1. Check the condition (i < 5).
2. If True, execute the block and update i.
3. Repeat until the condition becomes False. (When i = 5, the loop stops.)

2. Step-by-Step Execution



3. Trace Table (Execution Flow)

Iteration (#)	Value of i (before check)	Condition (i < 5)	Action
#1	0	True	repeat
#2	1	True	repeat
#3	2	True	repeat
#4	3	True	repeat
#5	4	True	repeat
#6	5	False	stop

Summary

- The condition (i < 5) is checked before each iteration.
- The block is executed while the condition is True.
- When the condition becomes False, the loop stops and the program continues.

while Statement: Printing a Multiplication Table

```c
#include < stdio.h >

int main( void )
{
    int n;
    int i = 1;

    printf ( "The number you want to print: " );
    scanf ( "%d" , &n);

    while ( i <= 9)
    {
        printf ( "%d*%d = %d \n" , n, i , n*i );
        i++;
    }
    return 0;
}
```

```
Enter the number you want to print : 9
9*1 = 9
9*2 = 18
9*3 = 27
....
9*9 = 81
```

Tips

- **if vs. while**
  - if (condition) { ... } is executed only ONCE when the condition is met.
  - while (condition) { ... } is executed repeatedly, multiple times, as long as the condition is met.

- **Conventional form**

```
while (i != 0)
{
   ...
}
```



```
while ( i )
{
   ...
}
```

- **Errors**



Beware of Errors

If you use a semicolon (;) at the end of the while condition, only NULL statements are repeated.

It is said that

```
while (i < 10) ;
```

It is treated as one sentence and only this is repeated.

```
i++;
```

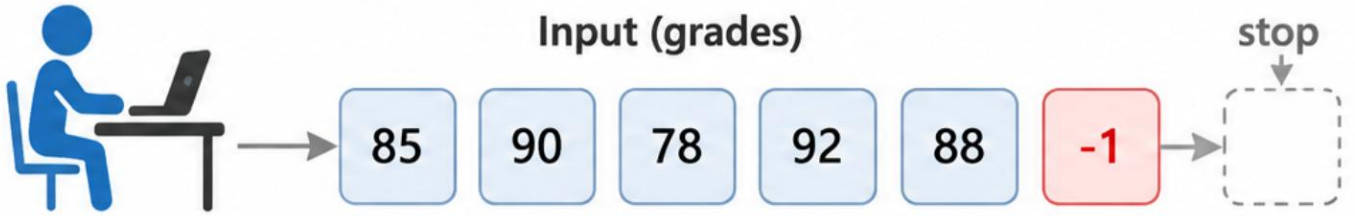
It doesn't repeat.

```
while (i = 2)
{
   ...
}
```

Since the value of the formula is 2, it is always true and the reason we fall in a loop.

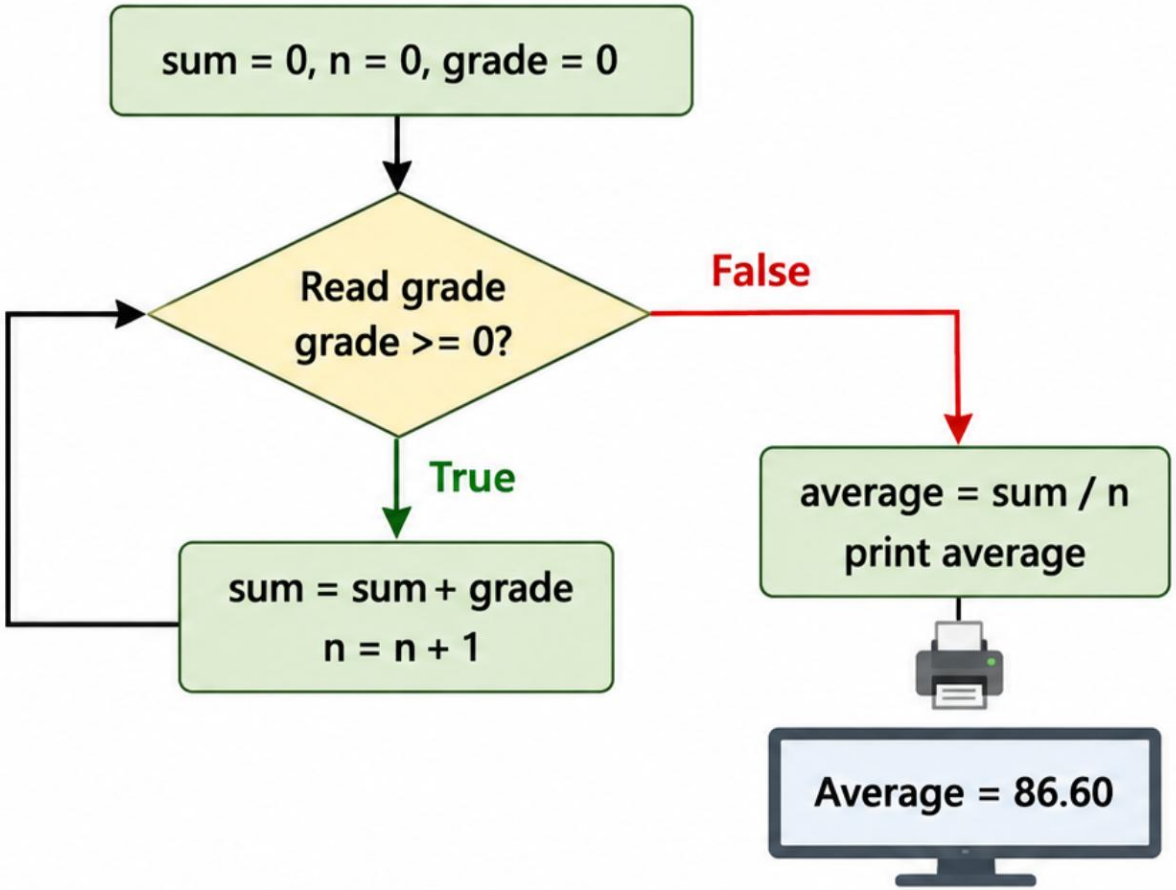
Sentinel-Controlled Loops

- **Sentinel:** a special value that signals the end of input data.



- **Problem:** write a program that receives an arbitrary number of grades from the user, calculates their average, and prints it. (using -1 as the sentinel value.)

1. **Initialize** the required variables (sum=0, n=0, grade=0).
2. **While** the score is not less than 0: read the grade, accumulate it in sum, and increase n by one.
3. Divide sum by n to compute the average, and print it.



do ~ while Loop

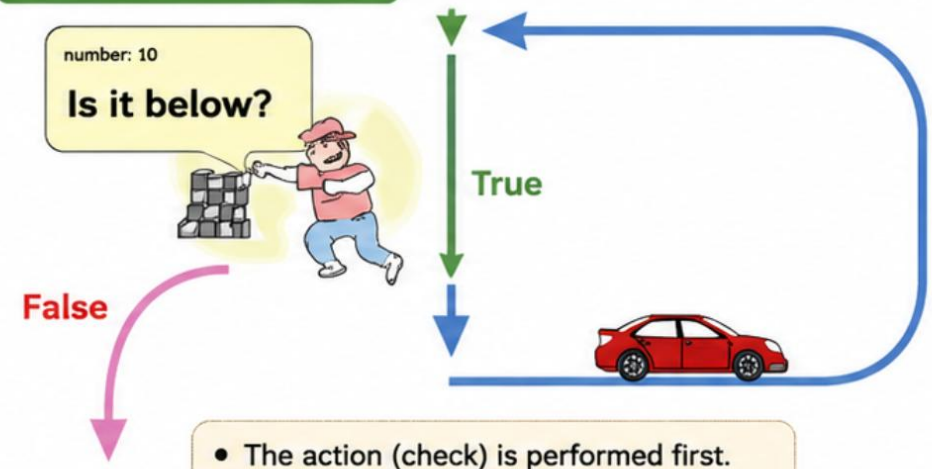
Syntax — do...while statement

```
do {
    statement 1;
    statement 2;
    ...
} while (condition);
```

Repetitive statements

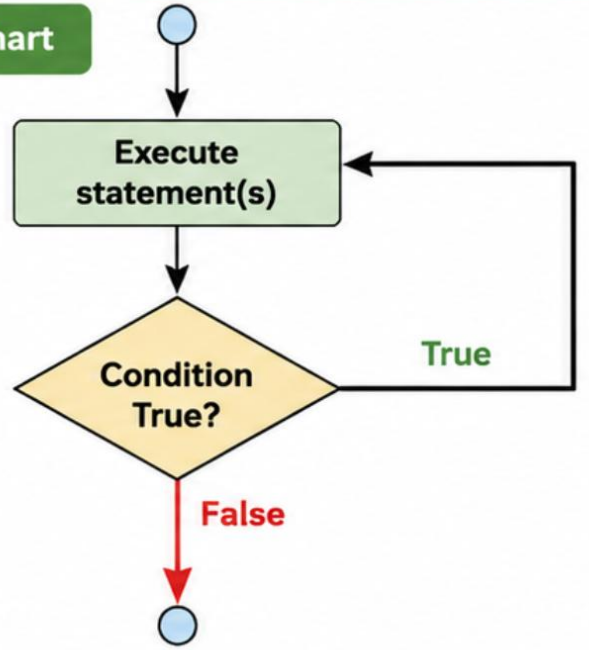
If the condition is True, the loop continues.
If the condition is False, the loop stops.

Real-world Example



- The action (check) is performed first.
- Then the condition is checked.
- Repeat while the answer is True.

Flowchart



- The statements are executed at least once.
- After execution, the condition is checked.
- If True, repeat.
- If False, stop.

Example: do-while Statement

```c
// Add numbers until the user enters 0.
#include < stdio.h >
int main( void )
{
    int number, sum = 0;

    // The loop body is executed at least once.
    do
    {
        printf ( "Enter an integer: " );
        scanf ( "%d" , &number);
        sum += number;
    } while (number != 0);

    printf ( "Sum of numbers = %d \n" , sum);
    return 0;
}
```

do-while executes the loop body at least once, checking the condition AFTER each pass - great for menus and input validation.

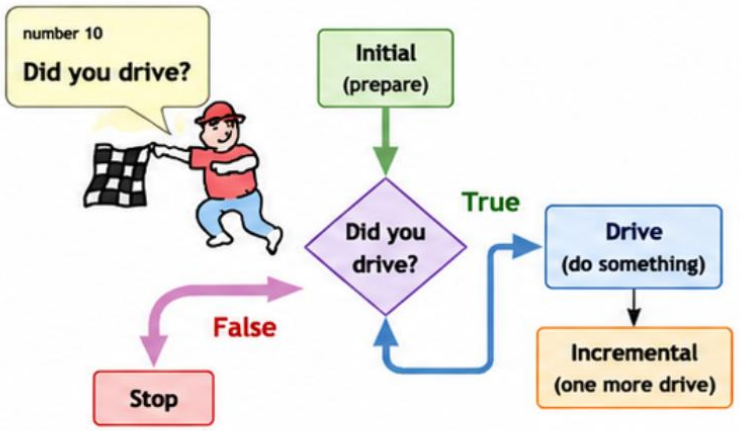
for Loop

Syntax — for statement

```
for ( initial ; condition ; incremental ) {
        statement 1;
        statement 2;
        ...
}
```

- **Initial** — Executed once at the beginning.
- **Conditional expression** — Evaluated before each iteration. If True, the loop continues. If False, the loop stops.
- **Incremental** — Executed after each iteration (before the next condition check).
- **Repetitive statements** — Executed repeatedly while the condition is True.

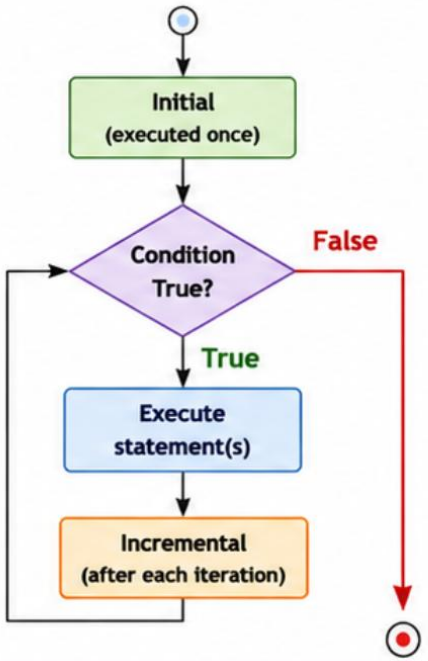
Real-world Analogy



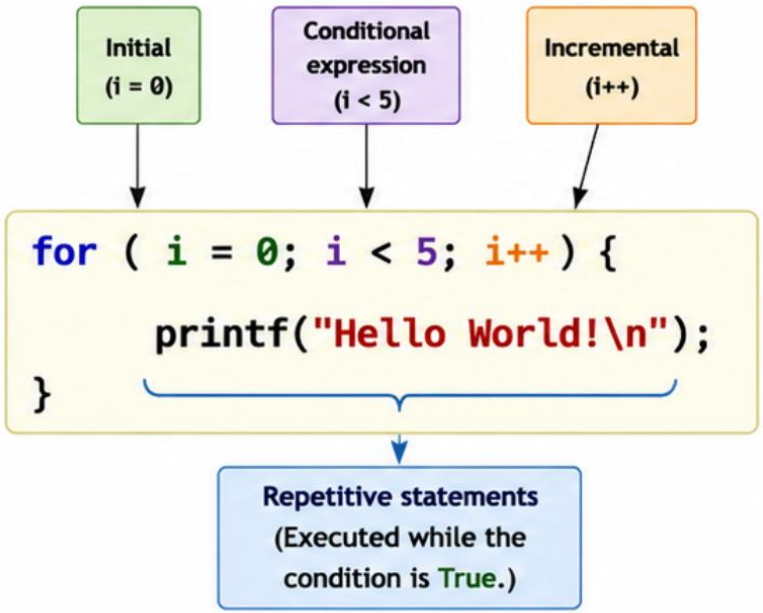
Initial | Conditional expression | Repetitive statements | Incremental | Stop

- Do the initial step.
- Check the condition.
- If True, do the task and increment.
- Repeat until the condition becomes False.

Flowchart



for statement structure



Example: The following loop prints "Hello World!" five times.

Output:

```
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
```

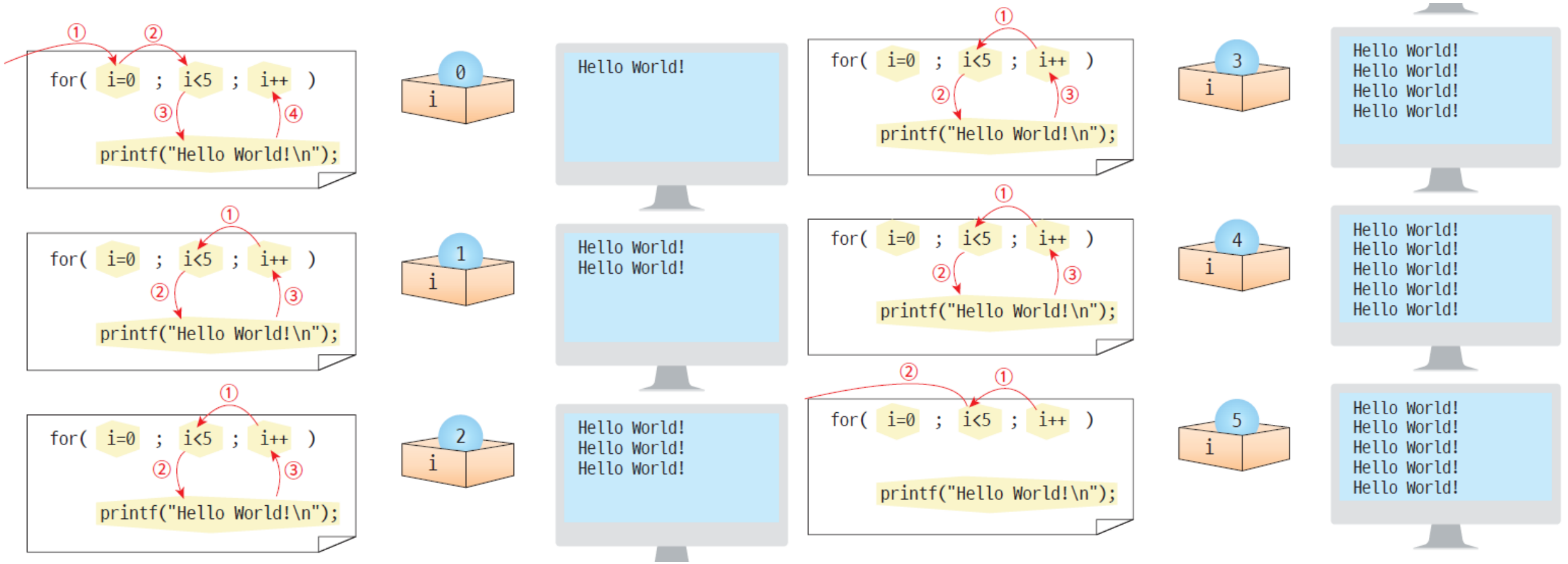
(i = 0, 1, 2, 3, 4)

for Loop: Initial, Conditional, and Incremental Expressions

- for (initial; condition; increment) { ... }
  - a structure that repeats a set number of times.
- Initial **expression**:
  - executed only once before starting the loop
  - mainly used to initialize variable values.
- Conditional **expression**:
  - checks the condition for repetition. If it becomes false, the repetition stops.
- Incremental **expression**:
  - executed after each loop pass completes.

for Loop: Execution Process

- for (i = 0; i < 5; i++) {
  printf("Hello World!\n");
}



Example: Sum from 1 to 10 with for loop

```c
// Integer sum program using repetition
#include <stdio.h>

int main( void )
{
    int i, sum;
    sum = 0;
    for (i = 1; i <= 10; i++) {
        sum += i; // same as sum = sum + i;
    }
    printf( "Sum of integers from 1 to 10 = %d\n" ,sum);
    return 0;
}
```

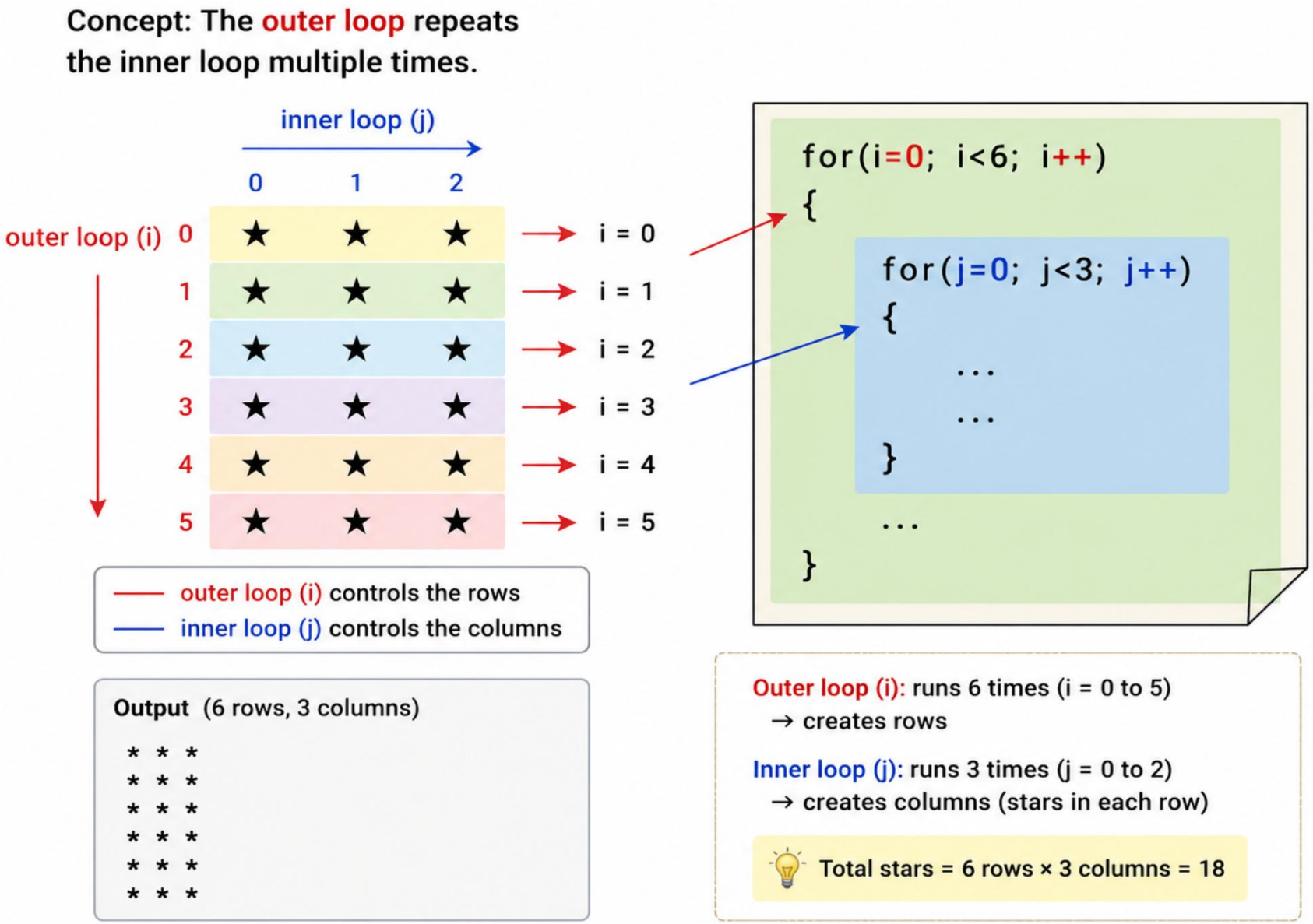
Various Forms of for Loops

- for (int i = 10; i > 0; **i--**) : counting down, using subtraction
- for (int i = 0; i < 10; **i += 2**) : increase by 2
- for (int i = 1; i < 10; **i *= 2**) : multiply by 2 each time - any formula is possible
- for (; i < 10; i++) { ... } : One part omitted
- for (**; ;**) { ... } : an **infinite loop** (all three parts omitted)
- for (**i = 0, j = 0; (i < 10 || j < 20); i++, j++**)
  : Multiple variables can be in all expression
- for (i = 0; (**i < 100 && sum < 2000)**; i++)
  : any complex formula can be a conditional expression

Nested Loops

- Nested loop:

a loop that is located within another loop.



Nested Loop Example: A Triangle of Stars

Practice

```
#include < stdio.h >
int main( void )
{
    int x, y;
    for (y = 0; y < 5; y++)
    {
        for (x = 0; x < 10; x++)
            printf ( "*" );
        printf ( "\n" ); // once per row
    }
    return 0;
}
```

```
**********
**********
**********
**********
**********
```

y stars are printed on the y-th line.

```
#include < stdio.h >
int main( void )
{
    int x, y;

                ?

    return 0;
}
```

```
*
**
***
****
*****
```

break and continue

Practice

- Infinite loop: sometimes intentional (e.g. a traffic light control program), but usually a bug you must **break** out of.

- **break**: used to exit a repeating loop immediately.
- **continue**: skips the rest of the current pass and jumps to the next iteration.
- **goto**: used to move to a label immediately.

- Continue example: print all integers from 0 to 10, EXCLUDING multiples of 3.

```c
while (1) {
    printf("Enter a real number: ");
    scanf("%lf", &v);
    if (v < 0.0)
        break;   // exit infinite loop
    printf("sqrt = %f\n", sqrt(v));
}
```

```c
// print all integers from 0 to 10, EXCLUDING
multiples of 3.

for (i=0; i<10; i++) {

                    ?

    printf("%d ", i);
}
```

Infinite loop

Syntax infinite loop statements

An **infinite loop** is a loop that never ends unless the program is terminated manually.

1) Infinite loop using while

Syntax

```
while (1) {
    if (condition)
        break;      # Stop repetition.
    if (condition)
        continue;   # Start the next iteration.
}
```

while (1)

- The condition is always true.
- The loop continues forever unless **break** is executed.

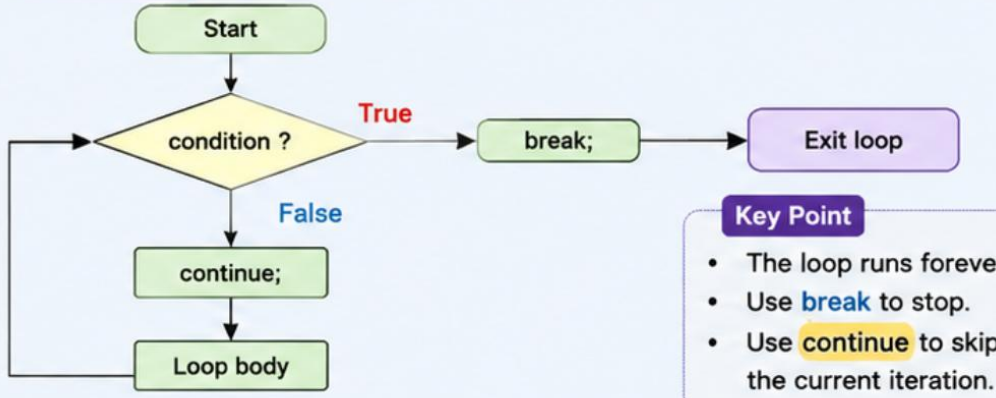
break

- Exits the loop completely.

continue

- Skips the rest of the code in the loop body and starts the next iteration.

Flowchart



Key Point

- The loop runs forever.
- Use **break** to stop.
- Use **continue** to skip the current iteration.

2) Infinite loop using for

Syntax

```
for (;;) {
    if (condition)
        break;      # Stop repetition.
    if (condition)
        continue;   # Start the next iteration.
}
```

for (;;)

- All three parts are empty.
- There is no initialization, no condition, and no increment → runs forever unless **break** is executed.

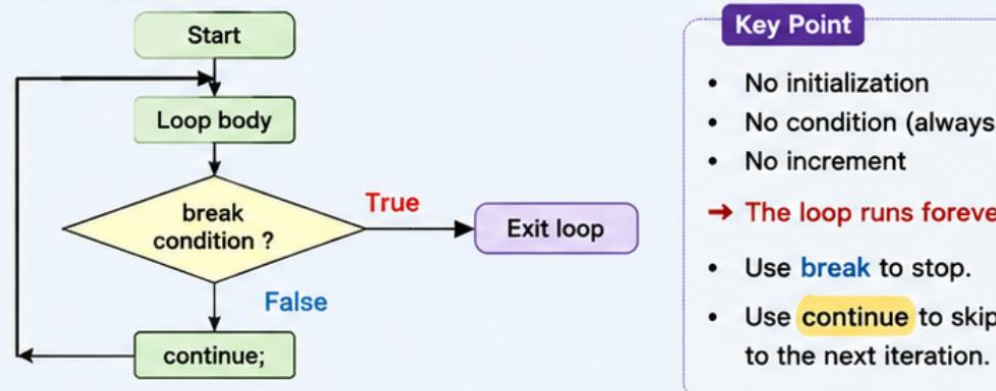
break

- Exits the loop completely.

continue

- Skips to the next iteration.

Flowchart



Key Point

- No initialization
- No condition (always true)
- No increment

→ The loop runs forever.

- Use **break** to stop.
- Use **continue** to skip to the next iteration.

Control Flow: break, continue, goto

break

Stop the loop completely.

```
while (condition) {
    ...
    if (some_condition)
        break;   // exit the loop
    ...
}
```

What it does

- Immediately terminates the nearest loop.
- The program continues with the statement after the loop.

Note (nested loops)

- **break** only exits the innermost loop it is inside.
- Outer loops continue.

continue

Skip the rest of this iteration and start the next one.

```
while (condition) {
    ...
    if (some_condition)
        continue;   // skip to next iteration
    ...
}
```

What it does

- Skips the remaining code in the loop body.
- Jumps to the next iteration (rechecks the condition).

goto

Jump to a labeled statement.

```
    ...
    if (some_condition)
        goto LABEL;   // jump to label
    ...
LABEL:
    statement;
  ...
```

What it does

- Transfers control to the labeled statement within the same function.
- Use with caution — it can make code harder to follow.

Summary

- **break:** Exit the nearest loop immediately. (In nested loops, only the innermost loop is exited.)
- **continue:** Skip the rest of this iteration and start the next iteration.
- **goto:** Jump to a labeled statement in the same function.

continue Statements

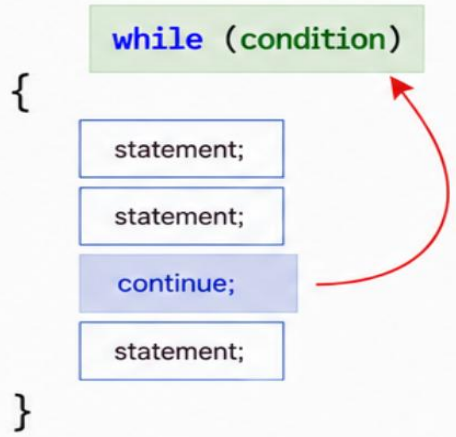
1) while Loop

```
while (condition) {
    // loop body
    ...
    continue;
}
```

continue;

When **continue;** is executed:

→ Go to the **while** (condition) to decide the next action.



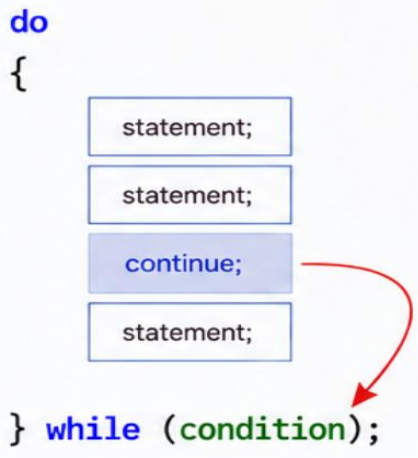
2) do-while Loop

```
do {
    // loop body
    ...
    continue;
} while (condition);
```

continue;

When **continue;** is executed:

→ Go to the **while** (condition) to decide the next action.



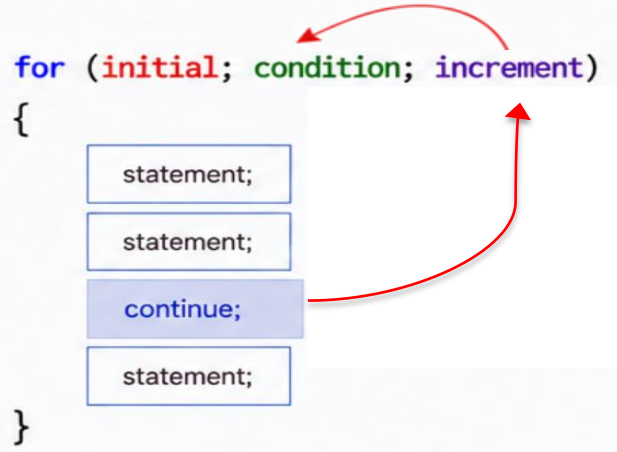
3) for Loop

```
for (initial; condition; increment) {
    // loop body
    ...
    continue;
}
```

continue;

When **continue;** is executed:

→ Go to the increment (increment) → then go to condition (condition) to decide the next action.



Q & A

Lab & Quiz: right after this - see you in the practice session!